Weekly Report

Ahmad Assaad

December 22, 2025

Abstract

The purpose of this report is to summarize the first week of my independent research. We will begin with the kinematics of the bicycle model. Then, we will follow it up with the pure pursuit algorithm. We will finally conclude with a future outlook and plan.

1 Introduction

This week has been mainly a broad literature review over the kinematic bicycle model, a geometric path following algorithm - pure pursuit, and neural networks. While the end goal or project has yet to be clearly specified, my work is heading towards adaptive learning algorithms for dynamic systems.

2 Kinematic bicycle model

2.1 Motivation

We first begin by discussing the motivation behind the kinematic bicycle model. We would like to model the motion of a vehicle; however, by removing a front and back wheel, we remove unnecessary complexity while keeping the model representative. Hence, the bicycle model represents a front wheel, which has a steering angle, and a back wheel, which does not have a steering angle, and they are connected through a rod-like structure.

Let us now discuss what the different parameters of this model convey:

We begin by defining the position of the rear wheel in the two–dimensional plane of motion. Let the rear and front radii, denoted R_r and R_f, represent the distances from each wheel to the instantaneous center of rotation (ICR). The distance between the rear and front wheels is given by L. Furthermore, we introduce the steering angle δ, which can be shown to be equivalent to the angle formed between the segments connecting the rear wheel to the ICR and the front wheel to the ICR.

Before developing the kinematic equations of the bicycle model, we must first introduce the concept of an *instantaneous center of rotation*. The ICR is the point about which the vehicle would undergo pure rotational motion at that instant, and it plays a central role in relating wheel geometry to vehicle motion.

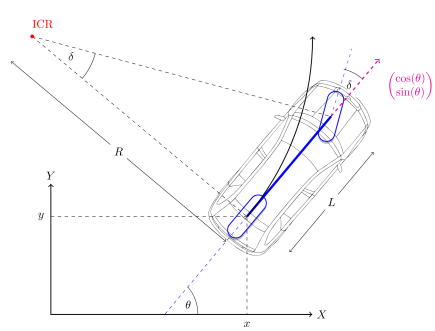

Figure 1: Visual representation of bicycle model

2.2 Instantaneous Center of Rotation (ICR)

In simple terms, it represents the center around which an object would undergo circular motion with that exact velocity. From this definition, we can derive useful equations in representing the velocity of different points based on their distance from the center as well as the angular velocity of the body. Geometrically, we know that the velocity vector is perpendicular to the trajectory as well as the radius; hence, my ICR will be the intersection of two lines perpendicular to the velocity vector at any two points.

Now, to see why this is useful to us, we first start with this equation:

$\mathbf{v}_B = \mathbf{v}_A + \boldsymbol{\omega} \times \mathbf{r}_{B/A}$

Taking A as my instantaneous center, then the equation becomes:

$\mathbf{v}_B = \boldsymbol{\omega} \times \mathbf{r}_{B/A}$

Hence, we now can derive the velocity, radius, or angular velocity of any point on the body.

2.3 Kinematic Equations

To represent the motion of the bicycle model we focus our attention on the rear wheel and not the front one (since focusing on the front wheel leads us to account for lateral slip (due to the geometry). To fully describe the motion of our bicycle model (assuming constant speed at first, if not we just include the velocity and acceleration in both the state and state dot vectors), we introduce state and state dot vectors (the incorporation of the vectors are just fancy ways of representing information).

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \\ v \end{bmatrix} \qquad \dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{v} \end{bmatrix}$$

Now, we derive the velocity along the x-axis as the velocity multiplied by the cosine of the heading angle, which is the yaw angle θ:

$$\dot{x} = v \cos\theta.$$

We derive the velocity along the y-axis in a similar fashion as the velocity multiplied by the sine of the heading angle:

$$\dot{y} = v \sin\theta.$$

Now, we are interested in finding an expression for the angular velocity of our model in terms of quantities we already know or can determine. Angular velocity is the rate of change of the heading angle:

$$\dot{\theta} = \omega.$$

We may obtain ω either by differentiating $\theta(t)$ or by using the relationship between velocity and turning radius. Considering the triangle formed by the ICR, the rear wheel, and the front wheel, and applying right–angle trigonometry, we find:

$$\omega = \frac{v \tan\delta}{L}.$$

Finally, the derivative of the velocity is the angular acceleration:

$$\dot{v} = a.$$

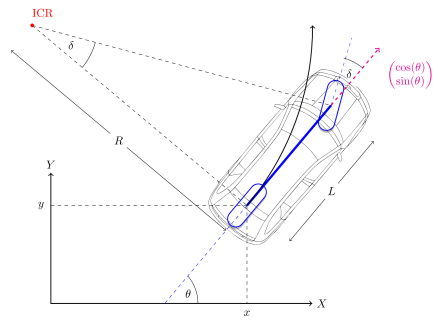

Figure 2: Visual of bicycle model

3 Pure Pursuit

3.1 Aim

After defining the kinematic bicycle model, we shift our focus towards path-following algorithms. Pure pursuit is the simplest of said algorithms as it is solely a geometric process. The question we then raise is this: by how much should I change my steering angle to follow a set target point? Our second question that follows is how should I set my target point for such an idea to work?

3.2 Logic

The idea behind the algorithm is simple: given the vehicle position and velocity, how much should the steering angle change in order to reach a target point on a pre-determined path. We have established that the bicycle model can be thought of as moving along a circular trajectory, and that changing the steering angle directly changes the curvature of this trajectory. By modeling the motion as circular and controlling its curvature, the problem reduces to relating the steering angle to the curvature of the desired circle (its radius and center).

1. Local frame of reference Up to this point, we have only used a global frame for the bicycle model. However, the pure pursuit algorithm requires the introduction of a local frame. Ideally, the vehicle heading is aligned with the local x-axis (forward/backward motion), while lateral motion lies along the y-axis (left/right). This is achieved through a linear transformation consisting of a translation and a rotation, with the vehicle's rear wheel chosen as the origin of the new coordinate system.

The transformation consists of:

- Translating the global frame by the rear wheel position.
- Rotating the frame by $-\psi$, where ψ is the vehicle yaw angle.

2. Target point definition To define the target point, two elements are required:

- **Lookahead distance**: the distance ahead of the vehicle used for tracking.
- **Path**: the desired path the vehicle should follow.

A constant distance from a point defines a circle. Representing the path as a polyline, the target point is defined as the intersection between the lookahead circle and the path segments. In the local frame, the lookahead circle is given by

$$x^2 + y^2 = L_d^2,$$

where L_d is the lookahead distance. Each path segment is represented parametrically as

$$x(t) = x_0 + t(x_1 - x_0), \quad y(t) = y_0 + t(y_1 - y_0),$$

with intersections restricted to $t \in [0, 1]$ to ensure forward motion and neglect backward solutions.

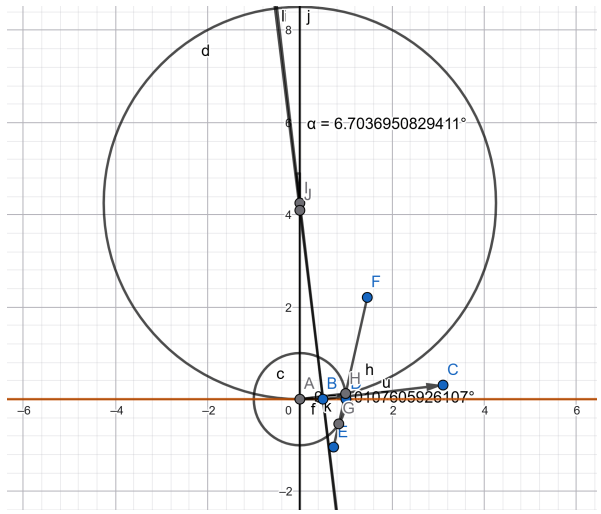

Figure 3: Depiction of pure pursuit

3. Steering angle computation The bicycle model is assumed to undergo constant circular motion, where the controlled quantity is the curvature (or equivalently, the radius). The steering angle δ is related to the radius R by

$$R = \frac{L}{\tan \delta},$$

where L is the wheelbase. To obtain an expression for R in terms of the target point, a geometric construction is used.

- A triangle is formed by the rear wheel, the target point, and the center of the circle passing through both points (given by the intersection of their perpendicular bisector with the local y-axis).
- The angle α is defined as the angle between the vehicle heading and the line connecting the rear wheel to the target point.
- The resulting triangle is isosceles. Using angle sum relations, the central angle is 2α. Applying the law of sines yields

  $$\frac{L_d}{\sin(2\alpha)} = \frac{R}{\cos \alpha}.$$

  Using the double-angle identity and simplifying gives the steering law

  $$\delta = \tan^{-1}\left(\frac{2L \sin \alpha}{L_d}\right).$$

4. State update The vehicle states are updated as

$$x_{k+1} = x_k + v_x \Delta t, \quad y_{k+1} = y_k + v_y \Delta t,$$

where the linear velocity magnitude remains constant. The angular velocity is given by

$$\omega = \frac{v}{R}.$$

5. Algorithm execution This procedure is repeated for all path segments in the polyline. The algorithm is non-adaptive, as the lookahead distance is fixed. A natural extension of this work would be to develop an adaptive lookahead strategy to minimize the tracking error between the vehicle position and the desired path.

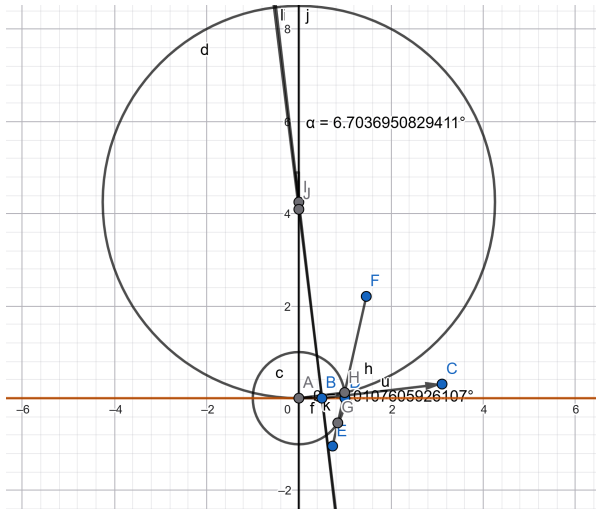

Figure 4: Low curvature path when steering angle is low

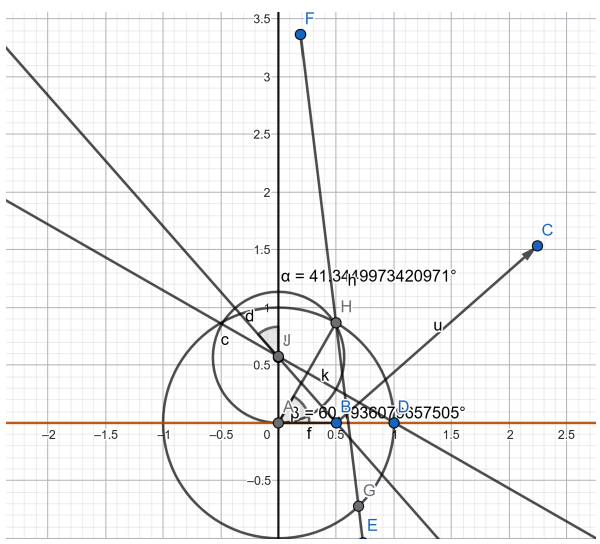

Figure 5: High curvature path when steering angle is high

6. Pseudocode

Algorithm: Pure Pursuit Path-Following

Input: Path polyline $\mathcal{P}$, lookahead distance L_d, wheelbase L, time step Δt
Initialize: Vehicle state (x, y, ψ), constant speed v

while vehicle is active **do**

1. Transform path to vehicle frame
$\mathcal{P}_{local} \leftarrow \emptyset$
for each $(x_p, y_p) \in \mathcal{P}$ **do**
$\Delta x \leftarrow x_p - x$
$\Delta y \leftarrow y_p - y$
$x_l \leftarrow \cos(-\psi)\Delta x - \sin(-\psi)\Delta y$
$y_l \leftarrow \sin(-\psi)\Delta x + \cos(-\psi)\Delta y$
append (x_l, y_l) to $\mathcal{P}_{local}$
end for

2. Target point selection
$p_{target} \leftarrow$
for each segment $(p_i, p_{i+1}) \in \mathcal{P}_{local}$ **do**
compute intersections with $x^2 + y^2 = L_d^2$
discard intersections with $x < 0$
if valid intersection exists **then**
$p_{target} \leftarrow$ first valid intersection
break
end if
end for

3. Steering computation
if $p_{target} \neq$ **then**
$\alpha \leftarrow \text{atan2}(y_{target}, x_{target})$
$\delta \leftarrow \tan^{-1}\left(\dfrac{2L\sin\alpha}{L_d}\right)$
else
$\delta \leftarrow 0$
end if

4. State update
$x \leftarrow x + v\cos\psi\,\Delta t$
$y \leftarrow y + v\sin\psi\,\Delta t$
$\psi \leftarrow \psi + \dfrac{v}{L}\tan\delta\,\Delta t$

end while

3.3 Conclusion and Outlook

This concludes the report on the kinematic bicycle model and the pure pursuit algorithm. While it serves as a good introduction, my work should now focus on path following algorithms as a whole as I am starting a new survey paper. Alongside that, I plan to build my intuition in adaptive and learning based methods through both neural networks and reinforcement learning, specifically algorithms in reinforcement learning and the impact of physics information within.

References

[1] T. Fermi, *Pure Pursuit*, Algorithms for Automated Driving, Available at: https://thomasfermi.github.io/Algorithms-for-Automated-Driving/Control/PurePursuit.html, Accessed: January 22, 2025.